Deep Learning for Visual Computing
Report: Assignment 1

Klemens Wiesinger [11938253], Vinzent Beer [12410094]

2026-05-05

0.1 Note on length

We are sorry about the document length, but we're just going to pretend that graphs and tables do not contribute to the length of the report, since otherwise it's really hard to answer all questions and display visualizations in 4 pages (and we do not want to make you flip between the body and appendix) :)

1 Questions

1.1 What is the purpose of a loss function? What does the cross-entropy measure? Which criteria must the ground-truth labels and predicted class-scores fulfill to support the cross-entropy loss, and how is this ensured?

- **Purpose:** A loss function measures how well a model's predictions match the ground truth. During training, the optimizer aims to minimize this loss, thereby improving the model's performance.
- **Cross-entropy** measures the dissimilarity of two probability distributions (in our case, the ground truth and the predicted distribution). It is high when the model assigns a low probability to the correct class and low when it assigns a high probability to the correct class.
- **Criteria:** both the ground truth labels and the predicted distribution must be valid probability distributions, meaning that all labels/predictions for one image need to sum up to 1 and need to be ≥ 0. In practice, the labels are often encoded as one-hot vectors (e.g., [0,0,1,0]), and the model's output logits are transformed into probabilities using the softmax function.

1.2 What is the purpose of the training, validation, and test sets, and why do we need all of them?

- **Training set:** The data set that the model actually trains and thus learns its parameters on.

- **Validation set:** The set we use to tune hyperparameters, model architecture. It gives an estimate of how the model generalizes to unseen data
- **Test set:** This is the set we evaluate the final performance on, after all training and tuning is finished. It should not be used to train or adapt the model in any way.
- **Why we need all three:**
  - Without a validation and a test set, we are prone to overfitting on the training
  - Without a validation set, we are prone to overfitting our hyperparameters on the test data (impacting real generalization)
  - Without a test set, we lack an unbiased way of assessing our model's performance

1.3 What are the goals of data augmentation, regularization, and early stopping? How exactly did you use these techniques (hyperparameters, combinations) and what were your results (train, val and test performance)? List all experiments and results, even if they did not work well, and discuss them.

- All three have the shared goal of improving model generalization by preventing overfitting.
- We use data augmentation to increase the variability in the available data without actually having to procure a larger dataset, and to use each sample available more efficiently. The same image when transformed can be another training datapoint, and prevent overfitting on specifics like location of the object in the image by flipping or randomly cropping it.
- Regularization adds constraints or other types of "obstacles" to the training routine to force the model to find generalizable solutions. One example is dropout, which we used in our CNN model between the two fully connected layers to prevent overfitting. It randomly disables a certain percentage of neurons during training.
- Early stopping is similar, it stops training when a specified metrics stops improving, since after that point models tend to just overfit on the training data to get the last few bits of performance / accuracy.
- For the experiments we stuck to a basic configuration of a batchsize of 128, validation frequency of 5, 30 epochs, ExponentialLR as lr scheduler with gamma = 0.9, AdamW with amsgrad=True and lr=0.001 as optimizer and CrossEntropyLoss
- We experimented with different data augmentation techniques, as can be seen in Figure 1. Combined usage had the biggest effect, but random cropping alone made for the biggest individual per class accuracy gains. In the base run, we can see a classic case of overfitting in the val_loss graph, since the model gets more and more confident in its wrong classifications over time, while using data augmentation and weight decay fixes this.

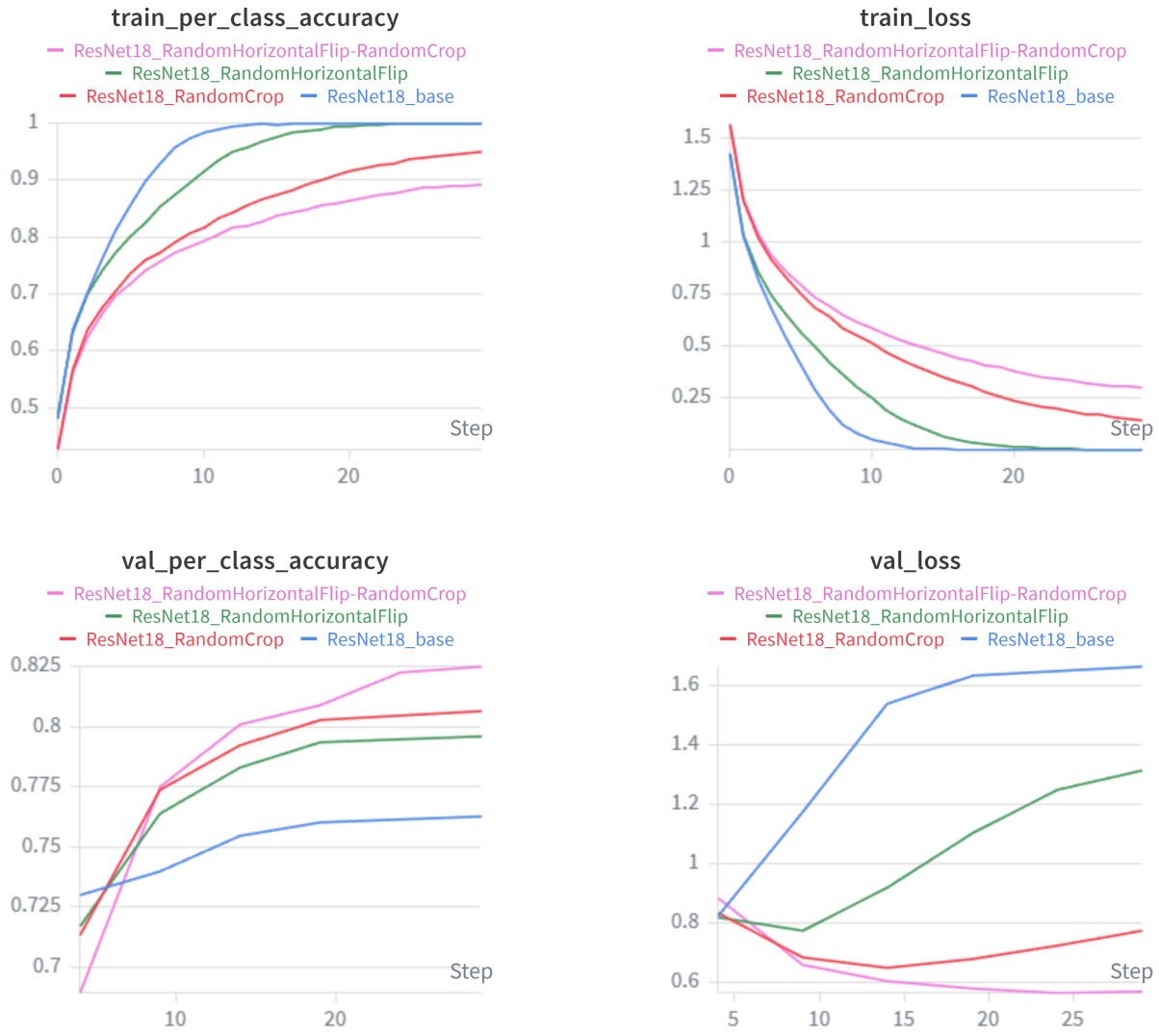

Figure 1: Experiments with different data augmentation

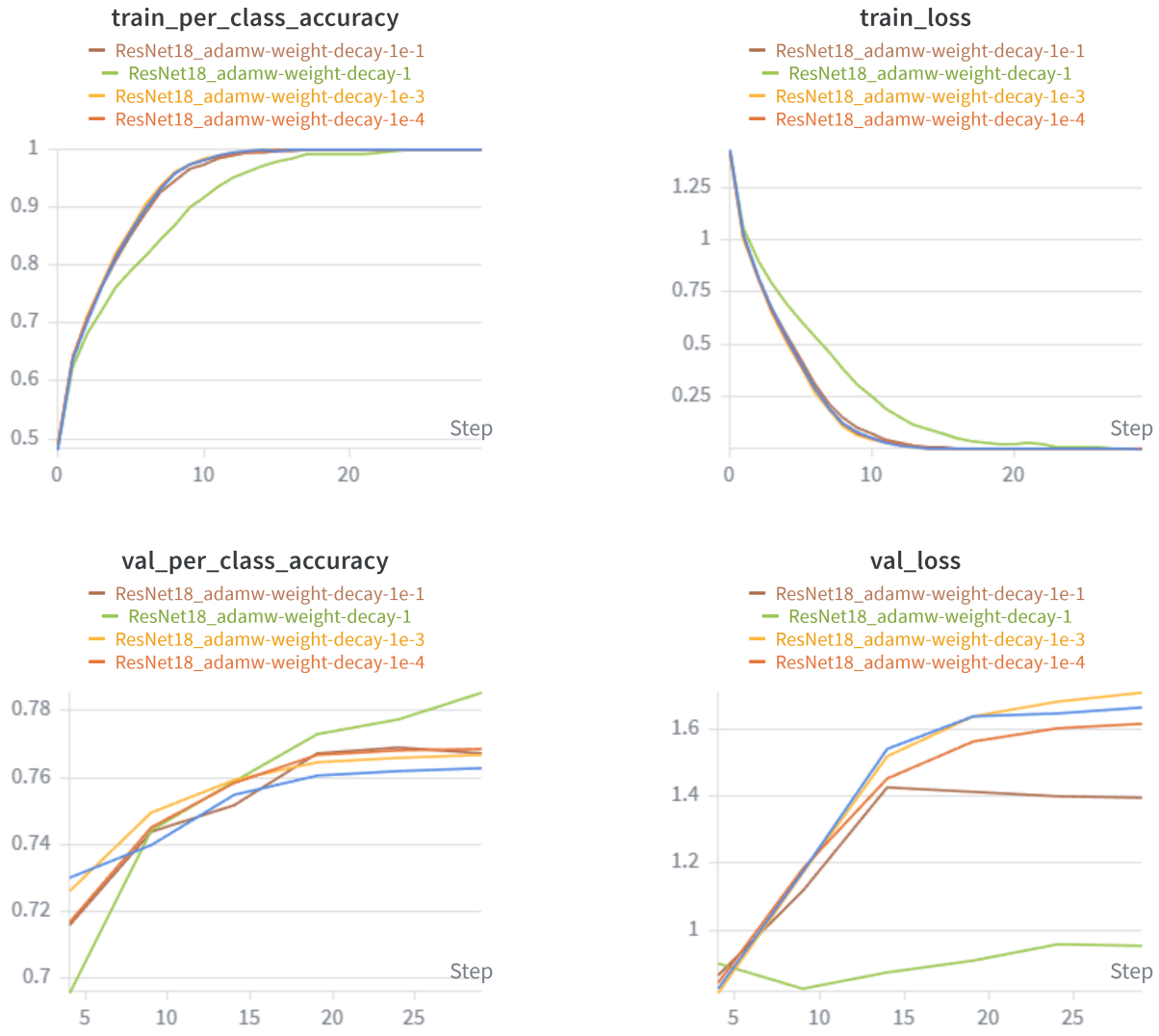

Figure 2: Experiments with different weight decay settings

In Figure 2 we experimented with weight decay. We had to go really bold, with values of up to 1, to get the best results. This comes as a surprise, as common knowledge dictates extremely careful usage of it and a value of 1 is much higher than anything we expected to be useful.

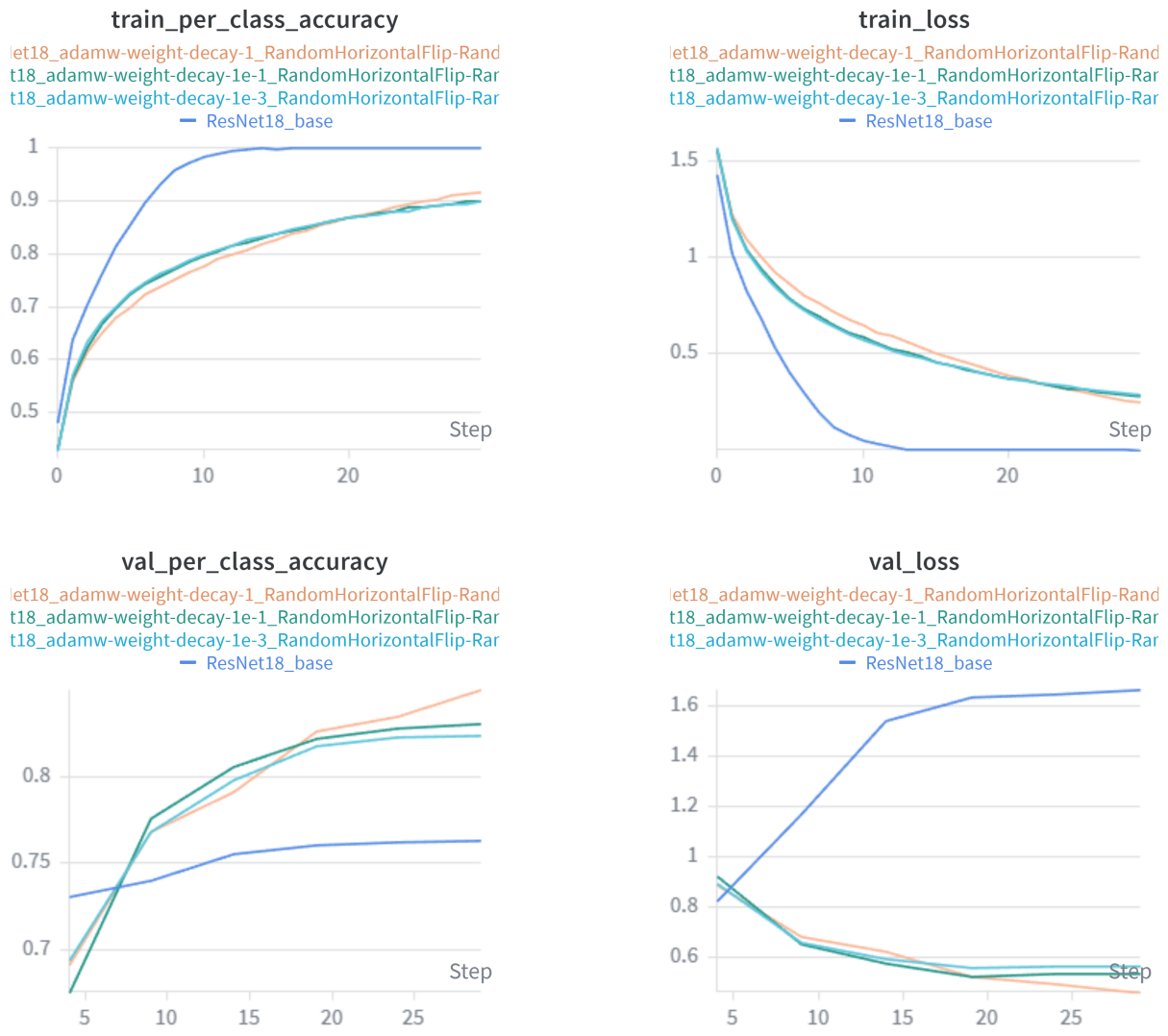

Figure 3: Experiments with combinations of weight decay and data augmentation

In the experiments represented by Figure 3 we experimented with combinations of weight decay and data augmentation. The combination of both produced slightly improved validation results than the individual applications.

Method	**Loss**	**Acc**	plane	car	bird	cat	deer	dog	frog	horse	ship	truck
Base	1.675	0.762	0.822	0.844	0.676	0.604	0.736	0.619	0.821	0.788	0.880	0.828
Augmentation	0.566	0.821	0.868	0.884	0.757	0.651	0.829	0.709	0.867	0.847	0.906	0.890
Weight Decay	0.967	0.784	0.832	0.875	0.706	0.556	0.794	0.693	0.829	0.835	0.877	0.840
WD + Aug	**0.480**	**0.848**	0.897	0.910	0.797	0.714	0.835	0.715	0.910	0.904	0.919	0.882

Table 1: Comparison of different training strategies on test performance.

In Table 1 we ran the best models from each experiments on the test data. Just like in the graphs,

we can see that Weight Decay + Augmentation combined provided the best loss and accuracy. When looking at the per-class-accuracies, we can see that certain classes seem easier to distinguish than others. Cat and dog sit at the bottom of the pack throughout, while ships seems easiest to clearly recognize.

1.4 What are the key differences between ViTs and CNNs? What are the underlying concepts, respectively? Give the source of your ViT model implementation and point to the specific lines of codes (in your code) where the key concept(s) of the ViT are and explain them.

- **CNNs** process images using convolution with shared local filters, enforcing translation-equivariant, local interactions. By stacking layers, they build hierarchical feature representations from simple to complex patterns, often combined with pooling for spatial aggregation.
- **ViTs** treat images as sequences of embedded patches and use self-attention to model global, data-dependent interactions between all patches. Positional encodings are added to retain spatial information.
- The key difference is that CNNs rely on fixed, local connectivity, whereas ViTs use global, dynamic connectivity, making locality inherent in CNNs but learned in ViTs.
- Our ViT is based on `https://lightning.ai/docs/pytorch/stable/notebooks/course_UvA-DL/11-vision-transformer.html`
- The key concepts:
  - **Patch embedding (tokenization)**: We first split the image into patches (line 105) using the `img_to_patch(...)` function. These patches are then projected into an embedding space using a linear layer (line 107). This transforms each patch into a fixed-dimensional vector, forming the input sequence to the transformer.
  - **Positional encoding and CLS token**: A learnable classification token (`cls_token`, line 100) is prepended to the sequence (lines 110–111). During self-attention, this token interacts with all patches and aggregates global information, and its final representation is used for classification (lines 120–121). Additionally, learnable positional embeddings (`pos_embedding`, line 101) are added element-wise to all tokens (line 112) to encode spatial information, since self-attention itself does not capture the order or position of patches.
  - **Attention-Block**: The transformer consists of multiple stacked attention blocks (line 93), each implemented in the `AttentionBlock` class. Each block first applies layer normalization (line 53), followed by multi-head self-attention (line 54), where each token attends to all other tokens and updates its representation as a weighted combination of them. A residual connection adds the original input to the attention output. This is followed by a feed-forward network (defined in lines 44 - 50, applied in line 55), which introduces non-linearity and further transforms the features, again combined with a residual connection.

2 Comparison of best models

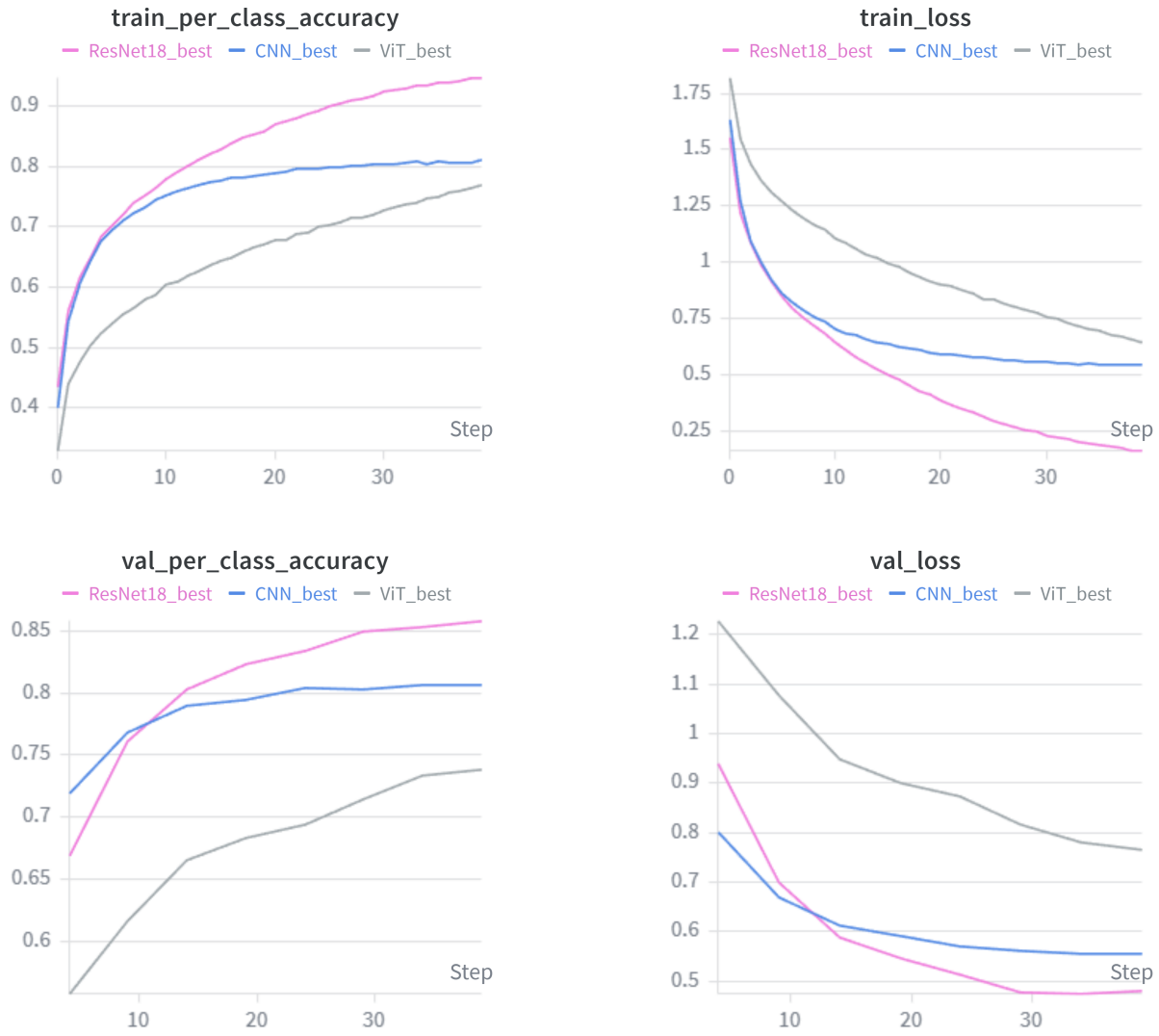

Figure 4: Comparison between the best iteration of each model

In our implementations, the CNN model performed much better than the ViT, even when it trained for longer. We suspect this is due to the relatively small dataset, even with augmentation. ViTs tend to require large amounts of input data and training time before they converge. However, it still lags behind ResNet18.

For the resnet18 model we used a batchsize of 128, validation frequency of 5, 40 epochs, ExponentialLR as lr scheduler with gamma = 0.9, AdamW with amsgrad=True and lr=0.001 as optimizer, CrossEntropyLoss, random horizontal flipping and random cropping for data augmentation, weight decay set to 1.

For the CNN model we used the same configuration, just without weight decay because we ran out

of time for testing it.

For the ViT we use the MultiStepLR scheduler with milestones = [100, 150] and gamma = 0.1, because that was what the tutorial used, again without weight decay.

Model	**Loss**	**Acc**	plane	car	bird	cat	deer	dog	frog	horse	ship	truck
ResNet	**0.494**	**0.848**	0.854	0.919	0.817	0.695	0.837	0.760	0.893	0.883	0.928	0.894
CNN	0.571	0.802	0.847	0.902	0.709	0.612	0.790	0.723	0.840	0.835	0.901	0.860
ViT	0.769	0.735	0.804	0.848	0.629	0.629	0.633	0.618	0.812	0.818	0.804	0.758

Table 2: Comparison of different model architectures on test performance.

In Table 2 we can see that unsurprisingly, ResNet performs the best on the test set as well.